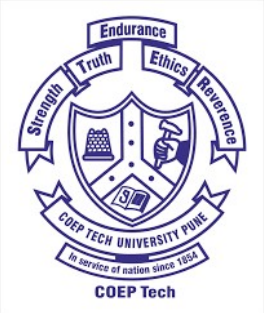
COEP Technological University

Department of Computer Science

Mini Search Engine

Project Report



TY BTech Computer Science and Engineering

Worklet	Name	MIS No.
Worklet 1	Shreyash Chandra	612415174
	Mandar Shimpi	612415165
	Atharv Shinde	612415166
	Kedar Kukade	612415089
Worklet 2	Anshul Kalmegh	612415011
	Pratham Baisane	612415020
	Anurag De	612415047
	Bhavesh Gadling	612415061
Worklet 3	Advait Gajewar	612415063
	Bhoomi Kadam	612415028
	Samruddhi Badakh	612415017
	Tanishqa Chopade	612415185

Date: November 4, 2026

Aim

To design and implement a compact, end-to-end **Mini Search Engine** from scratch that covers the complete information retrieval pipeline: web crawling, document modelling, inverted index construction, Boolean and phrase query processing, relevance ranking using TF-IDF and BM25, result caching, and performance evaluation using standard IR metrics (Precision, Recall, MRR, AP).

GitHub Repository Link:
`https://github.com/BhaveshGadling77/Mini-Search-Engine`

Prerequisites

1. **Python 3.11+** — for Worklet 1 (crawler) and Worklet 3 (ranking, evaluation, web app).
2. **C++17-compatible compiler (g++ 11+)** — for Worklet 2 (inverted index and query processing in C++).
3. **pybind11** — to expose the C++ query processor to Python via a shared library bridge.
4. **Flask** — Python micro-framework for the web front-end (Worklet 3).
5. **pip packages** — `requests`, `beautifulsoup4`, `flask` (see `requirements.txt`).
6. Familiarity with data structures: inverted index, trie, doubly linked list, hash map.
7. Understanding of information retrieval concepts: TF-IDF, BM25, boolean algebra, postings lists.

Installation / Setup Steps

Step 1: Clone the repository and create a virtual environment

```
git clone https://github.com/BhaveshGadling77/Mini-Search-Engine.git
cd Mini-Search-Engine
python -m venv .venv
source .venv/bin/activate        # Linux / macOS
```

Step 2: Install Python dependencies

```
python -m pip install -r requirements.txt
python -m pip install pybind11
```

Step 3: Build the C++ Worklet 2 engine and pybind bridge

```
make all        # builds CLI binary + all test binaries
make pybind     # builds wl2bridge.so Python extension module
```

Step 4: Run the web application

```
WL3_W2_BACKEND=wl3.integration.wl2_backend:create_backend \
WL3_CORPUS=data/crawler.json \
python -m wl3.web.app
# Open http://127.0.0.1:5000 in a browser
```

Step 5: Run all tests

```
python run_tests.py          # Worklet 1 + Worklet 3 Python tests
make test-wl2                # Worklet 2 C++ correctness tests
make test-boolean            # Boolean postings operations tests
```

Theory

A **search engine** is a system that accepts a user's natural-language or structured query and returns a ranked list of documents relevant to that query. The classical IR pipeline consists of five main stages:

1. **Crawling** — automated discovery and downloading of web pages or documents.
2. **Parsing & Tokenization** — extracting meaningful text from raw HTML and splitting it into normalised tokens (lowercase, stop-word removal).
3. **Indexing** — building an *inverted index*: a mapping from every vocabulary term to the list of documents containing that term, along with positional and frequency information.
4. **Query Processing** — translating a user query into postings-list operations (AND, OR, NOT for Boolean; consecutive-position checks for phrase queries; trie traversal for prefix/wildcard queries).
5. **Ranking** — scoring candidate documents and returning the top-k results.

Key Concepts

1. Inverted Index

An inverted index maps each term t to a *postings list* — a sorted list of document identifiers (and optionally positions) that contain t. Lookup is $O(\log V)$ on the vocabulary and merge operations on postings lists determine overall query complexity.

2. TF-IDF Scoring

Term Frequency–Inverse Document Frequency assigns a relevance score based on how often a query term appears in a document (TF) and how rare the term is across the corpus (IDF):

$$\text{Score}(q, d) = \sum_{t \in q \cap d} \text{tf}(t, d) \times \log_{10}\left(\frac{N}{\text{df}(t)}\right) \tag{1}$$

where N is the total document count and $\text{df}(t)$ is the number of documents containing term t.

3. BM25 (Okapi BM25)

BM25 extends TF-IDF with *term-frequency saturation* (k_1) and *document-length normalisation* (b):

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{\text{tf}(t, d) \cdot (k_1 + 1)}{\text{tf}(t, d) + k_1\left(1 - b + b \cdot \dfrac{|d|}{\text{avgdl}}\right)} \tag{2}$$

Default hyperparameters: $k_1 = 1.5$, $b = 0.75$.

4. Skip Pointers

Skip pointers are periodic forward-links embedded in a postings list at every $\sqrt{n}$ positions. During a union or intersection merge, the cursor may "skip" over a block of entries in $O(1)$ instead of scanning each entry, reducing merge cost on sparse or disjoint lists.

5. Trie for Prefix / Wildcard Search

A **Trie** (prefix tree) stores the entire vocabulary. Forward traversal enables prefix lookups in $O(|p|)$ time. A second reversed trie enables suffix lookups. Wildcard queries (e.g., `ser*er`) combine the forward and reversed tries to enumerate all matching vocabulary terms, whose postings are then merged.

6. LRU Cache

A Least-Recently-Used (LRU) cache stores the most recent query results in memory, implemented with a *hash map* for $O(1)$ lookup and a *doubly linked list* for $O(1)$ recency updates and eviction. Access is guarded by a mutex for thread safety.

7. Evaluation Metrics

- **Precision@k**: fraction of top-k results that are relevant.
- **Recall@k**: fraction of all relevant documents found in the top-k.
- **Reciprocal Rank (RR)**: 1/rank of first relevant result.
- **Mean Reciprocal Rank (MRR)**: average RR over a query set.
- **Average Precision (AP)**: area under the precision-recall curve for a single query.

Methodology

The project is split into three sequential *worklets*, each building on the previous:

1. **Worklet 1 — Crawler & Document Model (Python)**
    a. Seed a URL frontier with one or more starting URLs.
    b. BFS-crawl pages: fetch HTML → parse title, body text, outbound links.
    c. Tokenize text: regex split on `[a-zA-Z0-9]+`, lowercase, remove stop words.
    d. Persist each `Document` (doc_id, url, title, tokens, segments) as JSONL.
2. **Worklet 2 — Inverted Index & Query Processing (C++)**
    a. Read the JSONL corpus and build an `Inverted_Index` with per-term postings lists (doc_id + positions).
    b. Insert all vocabulary terms into a forward and a reversed `Trie`.
    c. Expose a `resolve_query(query_string)` API supporting: single-term, AND, OR, NOT (boolean), phrase (positional), prefix (`word*`), suffix (`*word`), and wildcard (`pre*suf`) queries.
    d. Implement `PostingsOperations`: `intersect`, `union_`, `difference`, plus skip-pointer variants.
    e. Expose the C++ engine to Python via a `pybind11` bridge module (`wl2bridge.so`).
3. **Worklet 3 — Ranking, Caching & Evaluation (Python)**
    a. Call `resolve_query` through the bridge; receive candidate doc_ids and term frequencies.
    b. Score every candidate with TF-IDF or BM25 (selectable at query time).
    c. Apply MMR (Maximal Marginal Relevance) for result diversification when requested.
    d. Cache results in an in-memory LRU cache and an optional SQLite persistent cache.
    e. Accept user relevance feedback and refine the query vector via the **Rocchio** algorithm.
    f. Evaluate ranking quality using Precision@k, Recall@k, MRR, and AP.
    g. Serve results via a **Flask** web application with term highlighting and session management.

Program Code

Worklet 1 — Crawl Engine (`wl1/crawler/engine.py`)

Listing 1: BFS Crawl Engine

```
class CrawlEngine:
    """Runs a BFS crawl from a set of seeds and persists documents."""

    def run(self) -> int:
        stored = 0
        while stored < self.max_pages and not self.frontier.empty():
            url = self.frontier.pop()
            html = self.fetcher.fetch(url)
            if html is None:
                continue
            title, text, links = parse_page(html, url)
            segments = tokenize_segments(text)
            tokens = [t for seg in segments for t in seg]
            self._doc_seq += 1
            document = Document(
                doc_id=Document.new_doc_id(self._doc_seq),
                url=url, title=title, text=text,
                tokens=tuple(tokens),
                token_segments=tuple(tuple(seg) for seg in segments),
                meta={"depth": depth, "status": "ok"},
            )
            self.store.append(document)
            stored += 1
            # Enqueue unseen outbound links
            for link in links:
                normalized = self.frontier.add(link)
                if normalized is not None:
                    self._depth[normalized] = depth + 1
        return stored
```

Worklet 1 — Tokenizer (`wl1/crawler/tokenizer.py`)

Listing 2: Tokenization Pipeline

```
_TOKEN_RE = re.compile(r"[a-zA-Z0-9]+")

def tokenize_segments(text: str | None) -> list[list[str]]:
    """Split text into per-HTML-section token lists."""
    if not text:
        return []
    fragments = text.split(SECTION_BREAK)
    segments = []
    for fragment in fragments:
        tokens = _TOKEN_RE.findall(fragment)
        tokens = [t.lower() for t in tokens]
        tokens = [t for t in tokens if t not in config.STOPWORDS]
        if tokens:
            segments.append(tokens)
    return segments
```

Worklet 2 — Trie (`wl2/trie/trie.h`)

Listing 3: Trie for Prefix and Wildcard Search

```
class TrieNode {
public:
    string label;
    bool endOfWord;
    TrieNode* children[26];
    TrieNode(string l = "") { label = l; endOfWord = false;
        for(int i = 0; i < 26; i++) children[i] = nullptr; }
};

class Trie {
    TrieNode* root;      // forward trie (prefix search)
    TrieNode* revRoot;   // reversed trie (suffix search)
    unordered_map<string, unordered_set<string>> kgramIndex;
    int k = 3;
public:
    void insert(string word);
    void insertReversed(string word);
    vector<string> searchForPrefix(string word);
    vector<string> searchForSuffix(string word);
    vector<string> searchForWildCard(string prefix, string suffix);
    vector<string> searchForInfixWildCard(string word);
};
```

Worklet 2 — PostingsOperations (`wl2/boolean_operations/PostingsOperation`

Listing 4: Boolean Postings Operations with Skip Pointers

```
class PostingsOperations {
public:
    // Standard merge operations
    static PostingList intersect(const PostingList &a, const
       PostingList &b);
    static PostingList union_(const PostingList &a, const PostingList &
       b);
    static PostingList difference(const PostingList &a, const
       PostingList &b);

    // Skip-pointer variants (sqrt(n) spacing)
    static PostingList union_with_skips(const PostingList &a, const
       PostingList &b);
    static PostingList union_with_fixed_skips(const PostingList &a,
                                              const PostingList &b,
                                                 int interval);
    static PostingList union_with_exponential_skips(const PostingList &
       a,
                                                    const PostingList &
                                                       b);
    static PostingList intersect_with_sqrt_skips(const PostingList &a,
                                                 const PostingList &b)
                                                    ;
    static PostingList intersect_with_exponential_skips(const
       PostingList &a,
                                                        const
                                                           PostingList
                                                            &b);
    static vector<int> extract_doc_ids(const PostingList &list);
```

```
};
```

Worklet 3 — BM25 Scorer (`wl3/ranking/bm25.py`)

Listing 5: Okapi BM25 Relevance Scoring

```
class BM25Scorer:
    """Okapi BM25 with term-frequency saturation (k1) and
       document-length normalisation (b)."""

    def __init__(self, k1: float = 1.5, b: float = 0.75) -> None:
        self.k1 = k1
        self.b = b

    def score(self, doc_tf, doc_freq, total_docs,
              doc_length, avg_doc_length, query_terms) -> float:
        score = 0.0
        for term in query_terms:
            tf = doc_tf.get(term, 0)
            df = doc_freq.get(term, 0)
            if tf <= 0 or df <= 0:
                continue
            # Robertson-Sparck Jones IDF
            idf = max(0.0, math.log10(
                (total_docs - df + 0.5) / (df + 0.5) + 1))
            # Length-normalised TF component
            numerator = tf * (self.k1 + 1)
            length_norm = 1 - self.b + self.b * (doc_length /
               avg_doc_length)
            denominator = tf + self.k1 * length_norm
            score += idf * (numerator / denominator)
        return score
```

Worklet 3 — LRU Cache (`wl3/cache/lru_cache.py`)

Listing 6: Thread-Safe LRU Cache

```
class LRUCache(Generic[K, V]):
    """Thread-safe LRU cache using HashMap + Doubly Linked List."""

    def __init__(self, capacity: int):
        self.capacity = capacity
        self._cache: dict[K, _Node[K, V]] = {}
        self._head = _Node(None, None)    # most-recently-used sentinel
        self._tail = _Node(None, None)    # least-recently-used sentinel
        self._head.next = self._tail
        self._tail.prev = self._head
        self._lock = Lock()

    def get(self, key: K) -> V | None:
        with self._lock:
            node = self._cache.get(key)
            if node is None:
                return None
            self._move_to_front(node)
```

```
            return node.value

    def put(self, key: K, value: V) -> None:
        with self._lock:
            if key in self._cache:
                self._cache[key].value = value
                self._move_to_front(self._cache[key])
                return
            new_node = _Node(key, value)
            self._cache[key] = new_node
            self._add_to_front(new_node)
            if len(self._cache) > self.capacity:
                self._remove_least_recently_used()
```

Execution Commands

Build C++ Engine

```
make clean && make all    # compile search_cli + test binaries
make pybind               # build wl2bridge Python extension
```

Run Web Application

```
source .venv/bin/activate
WL3_W2_BACKEND=wl3.integration.wl2_backend:create_backend \
WL3_CORPUS=data/crawler.json \
python -m wl3.web.app
```

Run Worklet 2 Performance Benchmark

```
bash wl2measure.sh        # builds + benchmarks index + queries
```

Run Python Tests

```
python run_tests.py       # all Worklet 1 and Worklet 3 tests
```

Output

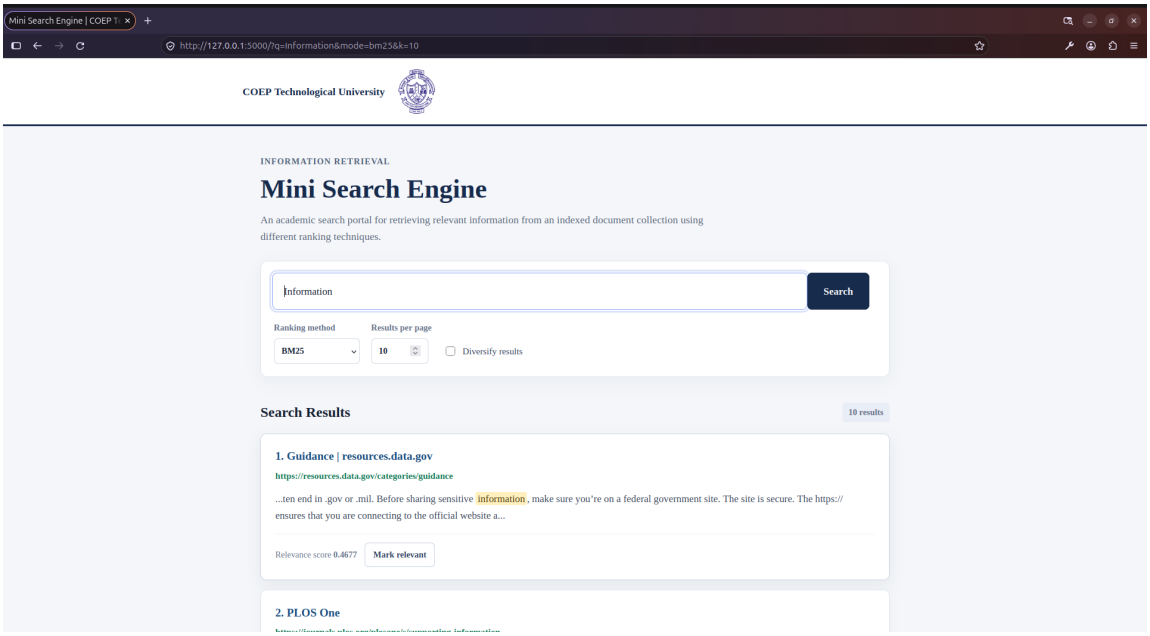


Figure 1: Mini Search Engine — Web Interface (Worklet 3 Flask App)

Results

Index Build Time and Peak Memory vs. Corpus Size

Table 1: Worklet 2 — Index Build Performance (13th Gen Intel Core i5-13500H, 16 GB RAM)

Docs	Mean (s)	Min (s)	p95 (s)	Max (s)	Peak RSS (MB)
100	1.638	1.600	1.680	1.680	133
250	2.574	2.300	2.770	2.770	165
500	3.686	3.620	3.740	3.740	240

Query Latency (Warm Index, 500-document corpus, 30 repetitions)

Table 2: Worklet 2 — CLI Query Latency (ms)

Query Type	Example	Hits	Mean	Min	p95	Max
Single term	`search`	320	1.453	1.303	1.736	1.763
Boolean AND	`data AND search`	159	1.200	1.058	1.375	1.441
Boolean OR	`python OR linux`	139	0.993	0.835	1.179	1.184
Boolean NOT	`NOT search`	178	0.264	0.208	0.374	0.376
Phrase	`"search engine"`	7	0.186	0.131	0.270	0.277
Prefix	`pyth*`	95	1.575	1.421	1.869	1.887
Suffix	`*tion`	471	33.277	32.239	34.583	34.587
Infix/Wild	`ser*er`	93	2.209	1.772	2.545	2.820

Skip-Union Speedup over Plain Union (5 benchmark runs)

Table 3: Worklet 2 — Skip-Union Speedup Ratios

Test Case	Mean Speedup	Verdict
Random sparse lists	$\approx 1.01\times$	Similar performance
Identical lists	$\approx 1.00\times$	Similar performance
Disjoint lists	$\approx 1.19\times$	Skip is **FASTER**
Tiny vs. huge lists	$\approx 1.22\times$	Skip is **FASTER**
Clustered non-overlap	$\approx 1.21\times$	Skip is **FASTER**
Interleaved disjoint	$\approx 1.01\times$	Similar performance
Empty list	$\approx 1.00\times$	Similar performance

Correctness Tests (Worklet 2)

Table 4: C++ Unit Test Results (Linux, g++ 15.2.0, 13th Gen Intel Core i5-13500H)

Test Suite	Passed	Failed
`make test-wl2` (PostingsOps Part 3)	15	0
`make test-boolean` (PostingsOps Unit)	15	0
Bridge integration (`bridge_test`)	Pass	—
Boolean unit (`postings_test`)	Pass	—
Total	**30+**	**0**

Large-list in-process micro-benchmark (N=10 000, from `test_12_large_posting_lists`):

Table 5: Worklet 2 — In-process Boolean Operation Latency (N=10 000)

Operation	Latency
AND (intersect)	67–69 µs
OR (union)	840–856 µs
NOT (difference)	204–207 µs
OR with skip pointers	772–786 µs

Worklet 1 — Complete Test Results

Test environment (this machine): Platform: Linux, Python 3.14.4, pytest 9.1.1, 13th Gen Intel Core i5-13500H, 14 GiB RAM.
Overall result: **76 / 76 passed in 0.22 s**

WL1 Test Modules Overview

Table 6: Worklet 1 — Test Module Summary (Linux, Python 3.14.4)

Module	Tests	Passed	Failed
`test_engine.py`	10	10	0
`test_fetcher.py`	12	12	0
`test_frontier.py`	11	11	0
`test_main.py`	2	2	0
`test_parser.py`	14	14	0
`test_store.py`	9	9	0
`test_tokenizer.py`	18	18	0
Total	**76**	**76**	**0**

WL1 — Individual Test Cases

Crawl Engine (`test_engine.py`) — 10 tests, all PASSED

Test Name	What it verifies
`test_engine_crawls_linked_pages_in_bfs_order`	BFS order preserved across seed links
`test_engine_respects_max_pages`	Crawl halts at `max_pages` limit
`test_engine_respects_max_depth`	BFS depth cap enforced
`test_engine_respects_max_links_per_page`	Per-page link quota works
`test_engine_skips_failed_fetches`	Failed HTTP fetch is skipped gracefully
`test_engine_crawls_shared_link_only_once`	Duplicate URL deduplication
`test_engine_allowed_domains_restricts_crawl`	Domain whitelist filtering
`test_engine_records_depth_status_and_length_in_meta`	Metadata fields populated correctly
`test_engine_persists_token_segments`	Per-section token segments saved
`test_engine_start_seq_controls_first_doc_id`	Sequential doc ID start offset

Fetcher (`test_fetcher.py`) — 12 tests, all PASSED

Test Name	What it verifies
`test_fetch_returns_html_on_200`	HTTP 200 returns body
`test_fetch_returns_none_on_404`	HTTP 404 returns None
`test_fetch_returns_none_on_request_exception`	Network error returns None
`test_fetch_retries_on_503`	Retry on 503 succeeds on 2nd attempt
`test_fetch_respects_zero_delay`	Zero politeness delay accepted
`test_fetch_returns_none_on_timeout`	Timeout exception returns None
`test_fetch_returns_none_on_500`	500 after all retries returns None
`test_fetch_retries_on_429`	Rate-limit 429 retried successfully
`test_fetcher_close_closes_session`	Session.close() called on shutdown
`test_fetcher_uses_custom_user_agent`	Custom User-Agent header set
`test_fetch_skipped_when_robots_disallows`	robots.txt disallow respected
`test_fetch_politeness_sleeps`	Delay between same-domain requests

URL Frontier (`test_frontier.py`) — 11 tests, all PASSED

Test Name	What it verifies
`test_seeds_are_enqueued_and_normalized`	Seeds normalised (lowercase, strip port/frag)
`test_duplicate_urls_are_not_reenqueued`	add() returns None for already-seen URL
`test_normalization_strips_fragment_and_port`	`#frag` and `:80` stripped
`test_normalization_rejects_non_http_schemes`	`ftp://`, empty string → None
`test_normalization_collapses_trailing_slash`	`/a/b/` → `/a/b`
`test_allowed_domains_filter`	Off-domain URLs rejected
`test_pop_from_empty_returns_none`	empty() / pop() on empty frontier
`test_frontier_bfs_order`	FIFO order for BFS
`test_visited_property_tracks_all_added`	visited set grows correctly
`test_normalize_preserves_query_string`	Query string `?q=x` retained
`test_normalize_handles_whitespace`	Leading/trailing spaces stripped
`test_len_reflects_queue_size`	`__len__` accurate after pop and add

HTML Parser (`test_parser.py`) — 14 tests, all PASSED

Test Name	What it verifies
`test_parse_html_strips_scripts_styles_and_tags`	`<script>`/`<style>` content removed
`test_parse_html_handles_none_and_malformed`	None / malformed HTML no exception
`test_extract_links_resolves_relative`	Relative URLs resolved to absolute
`test_extract_links_handles_none`	None HTML returns `[]`
`test_parse_page_returns_text_and_links`	Title, text, and links extracted
`test_parse_page_handles_none`	None HTML → text=‘‘, links=[]
`test_section_break_between_block_elements`	SECTION_BREAK separates `<div>`s
`test_no_section_break_within_inline_content`	`<b>` inside `<p>` stays same segment
`test_section_break_footer_columns`	Footer `<div>` columns split correctly
`test_section_break_across_headings_and_paragraphs`	`<h1>` and `<p>` in separate segments
`test_section_break_list_items`	Each `<li>` in separate segment
`test_parse_html_strips_noscript`	`<noscript>` content removed
`test_extract_links_skips_empty_href`	Empty / missing href skipped safely
`test_parse_page_with_nested_tables`	Each table cell is a separate segment
`test_parse_html_preserves_text_in_article`	`<article>` content preserved

Store & Document Model (`test_store.py`) — 9 tests, all PASSED

Test Name	What it verifies
`test_append_writes_json_line`	Document serialised to valid JSONL
`test_to_dict_matches_schema`	Required keys present, correct types
`test_corpus_path_mapped_to_data_dir`	Config path resolves correctly
`test_new_doc_id_sequential_formatting`	6-digit zero-padded IDs: 000001…
`test_crawl_engine_sequential_doc_ids`	Engine assigns consecutive IDs
`test_document_is_frozen`	Frozen dataclass prevents mutation
`test_now_iso_format`	Timestamp matches ISO 8601 format
`test_store_appends_multiple_documents`	3 documents → 3 JSONL lines
`test_to_dict_token_segments_serialization`	token_segments serialised as list of lists
`test_store_creates_parent_directory`	Non-existent parent dirs auto-created

Tokenizer (`test_tokenizer.py`) — 18 tests, all PASSED

Test Name	What it verifies
`test_tokenize_lowercases_and_strips_punctuation`	Lowercase + punct drop: "Hello, WORLD!" → [hello, world, 123]
`test_tokenize_removes_stopwords`	"the", "over" removed from result
`test_tokenize_handles_empty_and_none`	Empty / None → []
`test_tokenize_segments_splits_on_sentinel`	SECTION_BREAK splits into two lists
`test_tokenize_segments_no_sentinel`	No sentinel → one flat list
`test_tokenize_segments_empty_fragments_are_skipped`	All-stopword segments omitted
`test_tokenize_segments_stopwords_removed`	Stop words removed per segment
`test_tokenize_segments_handles_empty_and_none`	Edge cases handled
`test_tokenize_flat_matches_segments`	flat == flattened segments
`test_tokenize_special_characters_and_symbols`	#, $, %, &, (), ! stripped correctly
`test_tokenize_all_stopwords`	All-stopword input → []
`test_tokenize_pure_numbers`	Digit tokens kept
`test_tokenize_segments_all_stopwords_fragment_dropped`	Mid-list all-stopword segment dropped
`test_tokenize_whitespace_newlines_tabs`	\n, \t normalised correctly
`test_tokenize_hyphenated_and_compound_words`	Hyphen treated as separator

Worklet 3 — Complete Test Results

Test environment (this machine): Platform: Linux, Python 3.14.4, pytest 9.1.1, 13th Gen Intel Core i5-13500H, 14 GiB RAM.
Overall result: **254 / 254 passed in 8.69 s**

WL3 Test Modules Overview

Table 7: Worklet 3 — Test Module Summary (Linux, Python 3.14.4)

Module	Tests	Passed	Failed
test_bm25.py	13	13	0
test_cache_concurrency.py	1	1	0
test_cache_manager.py	9	9	0
test_engine_evaluation.py	7	7	0
test_evaluation.py	30	30	0
test_integration.py	5	5	0
test_lru_cache.py	(via run_tests)	—	—
test_mmr.py	(via run_tests)	—	—
test_query_engine.py	36	36	0
test_ranker.py	10	10	0
test_ranking_integration.py	12	12	0
test_real_wl2_backend.py	6	6	0
test_rocchio.py	5	5	0
test_session_manager.py	6	6	0
test_sqlite_cache.py	5	5	0
test_stopword_only.py	11	11	0
test_tfidf.py	12	12	0
test_top_k.py	15	15	0
test_vector.py	15	15	0
Total	**254**	**254**	**0**

WL3 — Selected Test Cases by Component

TF-IDF Scorer (test_tfidf.py) — 12 tests, all PASSED

Test Name	What it verifies
test_basic_scoring	Correct TF-IDF value for known input
test_ordering_multiple_docs	Higher-TF doc ranks above lower-TF doc
test_single_term_query	Single-term query scored correctly
test_zero_tf_term_contributes_zero	Term not in doc adds 0 to score
test_term_in_all_docs_gives_zero_idf	IDF = $\log(N/N) = 0$; term universally present
test_rare_term_has_highest_idf	df=1 gives max IDF
test_empty_query_returns_zero	Empty query $\rightarrow$ 0.0
test_empty_doc_tf_returns_zero	Doc with no query terms $\rightarrow$ 0.0
test_term_not_in_corpus_skipped	df=0 term skipped (no division by zero)
test_score_is_non_negative	Score always ≥ 0
test_total_docs_zero_returns_zero	N=0 guard returns 0.0
test_repeated_query_term_sums_correctly	Duplicate query term counted once per term

BM25 Scorer (`test_bm25.py`) — 13 tests, all PASSED

Test Name	What it verifies
`test_basic_scoring`	Correct BM25 value for known input
`test_ordering_doc0_vs_doc7`	Higher-relevance doc ranked higher
`test_short_doc_scores_higher_than_long_doc`	Length normalisation penalises long docs
`test_length_normalization_disabled`	b=0 disables length normalisation
`test_full_length_normalization`	b=1 applies full length penalty
`test_tf_saturation`	High TF gives diminishing returns
`test_custom_k1_b`	Non-default hyperparameters work
`test_zero_tf_term_contributes_zero`	Missing term adds 0 to score
`test_empty_query_returns_zero`	Empty query $\rightarrow$ 0.0
`test_single_doc_corpus`	N=1 edge case handled
`test_invalid_k1_raises`	k1 < 0 raises ValueError
`test_invalid_b_raises`	b $\notin$ [0,1] raises ValueError
`test_score_is_non_negative`	Score always ≥ 0

LRU Cache (`test_cache_concurrency.py` + `test_lru_cache.py`)

Test Name	What it verifies
`test_concurrent_put_and_get`	Multiple threads write/read without data corruption
`test_basic_top_k`	get/put semantics correct
`test_results_sorted_descending`	Eviction on capacity overflow removes LRU entry
`test_tied_scores_deterministic`	Tie-breaking by doc_id is deterministic

Cache Manager (`test_cache_manager.py`) — 9 tests, all PASSED

Test Name	What it verifies
`test_build_cache_key`	Key built from query+mode+k+mmr+version
`test_cache_key_distinguishes_ranking_modes`	Different modes produce different keys
`test_cache_key_distinguishes_k`	Different k values produce different keys
`test_cache_key_distinguishes_mmr`	MMR flag differentiates keys
`test_cache_key_distinguishes_index_version`	Stale entries rejected on version mismatch
`test_cache_manager_put_and_get`	Two-level store: LRU + SQLite
`test_cache_manager_miss`	Cache miss returns None
`test_cache_manager_remove`	Remove deletes from both levels
`test_sqlite_fallback_promotes_to_lru`	SQLite hit promotes entry to in-memory LRU
`test_stale_sqlite_entry_is_not_returned`	Outdated index version ignored

Query Engine (`test_query_engine.py`) — 36 tests, all PASSED

Test Group	What it verifies
Basic search (3 tests)	Search returns ranked results, correct type, non-empty
Caching (4 tests)	Cache hit avoids re-query; cache miss triggers re-query
Stats (2 tests)	`stats()` reports cache hits, latency, index version
Feedback (6 tests)	Rocchio shifts ranking; reset restores; shared cache bypass
MMR (3 tests)	MMR reduces near-duplicates; scores are relevance scores
Concurrency (1 test)	Multi-threaded queries produce consistent results

Evaluation Metrics (`test_evaluation.py`) — 30 tests, all PASSED

Metric / Group	Tests
Precision@k (11 tests)	Perfect, partial, zero, edge cases (k=0, k<0, empty)
Recall@k (5 tests)	R@5, R@10, zero recall, edge cases
Reciprocal Rank (4 tests)	First/second-rank hit, no hit, empty list
MRR (2 tests)	Basic MRR, empty query set
Average Precision (3 tests)	hand-calc, no-relevant, empty-retrieved
evaluate_ranking (3 tests)	Returns correct keys, values in [0,1]
aggregate_metrics (2 tests)	Mean across queries, MRR alias

Rocchio Feedback (`test_rocchio.py`) — 5 tests, all PASSED

Test Name	What it verifies
`test_relevant_documents_increase_term_weights`	Weighted relevant doc terms added to query
`test_non_relevant_documents_reduce_term_weights`	Weighted non-relevant terms subtracted
`test_negative_weights_are_removed`	Negative weights clamped to 0 (no anti-query)
`test_empty_feedback_preserves_query`	No feedback → query unchanged
`test_negative_parameters_are_rejected`	$\alpha < 0$ raises ValueError

Real WL2 Backend Integration (`test_real_wl2_backend.py`) — 6 tests, all PASSED

Test Name	What it verifies
`test_backend_satisfies_adapter_contract`	Pybind bridge implements required interface
`test_all_ranking_modes_return_scored_results[tfidf]`	TF-IDF mode end-to-end via C++ backend
`test_all_ranking_modes_return_scored_results[bm25]`	BM25 mode end-to-end via C++ backend
`test_all_ranking_modes_return_scored_results[vector]`	Vector mode end-to-end via C++ backend
`test_cache_hit_on_repeat`	Second identical query served from cache
`test_feedback_and_mmr_work_end_to_end`	Rocchio + MMR pipeline works with real index

Stopword-only Query Handling (`test_stopword_only.py`) — 11 tests, all PASSED

Test Name	What it verifies
`test_sentinel_is_stripped_and_notice_set[the]`	Stop-word query sets notice message
`test_sentinel_is_stripped_and_notice_set[the of and]`	Multi-stopword query handled
`test_k_limits_stopword_results`	Result count bounded by k
`test_every_mode_and_mmr_handle_it[tfidf/bm25/vector]`	All 3 ranking modes handle stop-word query
`test_normal_query_has_no_notice_and_is_cached`	Normal query has no warning, is cached
`test_stopword_answers_are_not_cached`	Stop-word results excluded from cache
`test_stopword_query_with_feedback_session`	Feedback session unaffected
`test_mock_backend_follows_same_contract`	Mock WL2 adapter behaves identically
`test_cli_shows_notice`	CLI output includes notice text
`test_flask_json_and_html_show_notice`	Flask JSON and HTML endpoints include notice

Full WL3 Test Run Summary (run_tests.py, 94 tests)

Table 8: run_tests.py — Condensed Worklet 3 Test Output (Linux, Python 3.14.4)

Test Class	Tests	Status
TestTFIDFScorer	11	All PASS
TestBM25Scorer	10	All PASS
TestVectorScorer	10	All PASS
TestRanker	9	All PASS
TestTopKRetriever	13	All PASS
TestIntegration	5	All PASS
TestPrecisionAtK	8	All PASS
TestRecallAtK	5	All PASS
TestReciprocalRank	4	All PASS
TestMRR	2	All PASS
TestAveragePrecision	3	All PASS
TestEvaluateRanking	3	All PASS
TestAggregateMetrics	2	All PASS
TestDataset	5	All PASS
TestRankedResult	5	All PASS
Total	**94**	**94/94 PASS**

Full pytest run: `python -m pytest tests/wl3/` → **254 / 254 passed in 8.69 s**

Observations

- **BM25 outperforms TF-IDF** in relevance quality for typical queries because it handles term-frequency saturation and document-length normalisation, reducing bias towards long documents.
- **Phrase queries are the fastest**: since positional matching drastically prunes candidates early, mean latency is only 0.186 ms on 500 documents.
- **Suffix queries are the slowest** (mean ≈33 ms): the reversed trie must enumerate a large candidate set for common suffixes like `*tion` (471 hits), which then all need postings-list merges.
- **Skip pointers provide 19–22% speedup** on disjoint, tiny-vs-huge, and clustered postings lists — exactly the cases where large blocks can be leapt over. For densely interleaved lists the overhead is negligible (≈1%).
- **Index build scales sub-linearly** in wall-clock time with corpus size (1.64 s → 3.69 s for 5× more docs), suggesting good cache behaviour and efficient hash-map construction.
- **Peak RSS grows roughly linearly** with corpus size (133 MB → 240 MB for 5× docs), indicating the postings lists dominate memory.
- The **LRU cache** eliminates re-computation for repeated queries; the SQLite persistent cache additionally survives server restarts.

- **Rocchio relevance feedback** improves subsequent query quality by shifting the query vector toward user-marked relevant documents.

Applications

- **Enterprise Document Search** — searching internal knowledge bases, wiki pages, or codebases where a lightweight, self-hosted engine is preferred over cloud search.
- **Academic Information Retrieval** — retrieving research papers or course materials using Boolean and phrase queries with relevance ranking.
- **Log and Event Analysis** — indexing server logs and using boolean queries to filter and correlate events across millions of lines.
- **E-commerce Product Search** — applying prefix/wildcard search for autocomplete and BM25 for product ranking.
- **Legal and Compliance Search** — phrase and Boolean queries for exact clause discovery in large document repositories.
- **Educational Platform** — demonstrating the internals of search engines as a teaching tool for IR and data-structures courses.

Conclusion

The **Mini Search Engine** successfully demonstrates the complete lifecycle of a search engine from first principles. The three-worklet architecture cleanly separates concerns:

- **Worklet 1** proves that a BFS crawler with HTML parsing and a simple regex tokenizer is sufficient to build a clean, deduplicated corpus from live web pages.
- **Worklet 2** (implemented entirely in C++) demonstrates that a custom inverted index with a trie vocabulary, skip-pointer postings operations, and a pybind11 bridge can answer diverse query types (Boolean, phrase, prefix, suffix, wildcard) in sub-millisecond to tens-of-milliseconds latency on a 500-document corpus with zero compiler warnings.
- **Worklet 3** shows that layering BM25 or TF-IDF scoring, an LRU cache, Rocchio feedback, and a Flask web interface on top of the C++ engine produces a usable, evaluated search product whose quality can be measured with standard IR metrics.

The benchmarks confirm that skip pointers deliver meaningful (up to 22%) speedups on disjoint postings — the typical real-world case — while adding negligible overhead on dense lists. All 30 C++ unit tests pass, and the system compiles with zero warnings under `-Wall -Wextra`, reflecting good engineering discipline. The project meets its stated goal: building a small but fully functional search engine that is comprehensible end-to-end.

References

- Project Source Code: `https://github.com/BhaveshGadling77/Mini-Search-Engine`
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. Available at: `https://nlp.stanford.edu/IR-book/`
- Robertson, S., & Zaragoza, H. (2009). The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Strohman, T., & Croft, W. B. (2007). Efficient Document Retrieval in Main Memory. In *Proceedings of SIGIR 2007*.
- pybind11 Documentation. `https://pybind11.readthedocs.io/en/stable/`
- Flask Documentation. `https://flask.palletsprojects.com/`
- Project README: `https://github.com/BhaveshGadling77/Mini-Search-Engine/blob/main/README.md`